

Практическое занятие 2: Docker в production

Практическое занятие 2: Docker в production — контейнеризация полнофункционального приложения

Длительность: 3 часа (теория по лекции «Docker: от базового использования к production» + практика). **Цель:** Научиться собирать и запускать контейнеры «production-качество»: multi-stage-сборка, build cache, .dockerignore, healthchecks, секреты, лимиты ресурсов, graceful shutdown и систематическая диагностика. Всё это применить на реальном приложении из трёх компонентов: фронтенд, бэкенд, база данных — плюс nginx-шлюз с self-signed TLS для всего стенда.

Связь с предыдущим занятием: в ПЗ 1 мы разобрали Ansible + Semaphore как способ автоматизировать серверы. Сейчас мы готовим «артефакты» (Docker-образы), которые CI/CD будет собирать и доставлять на серверы. С этого момента и дальше Kubernetes — только как один из вариантов оркестрации, а не центр курса.

Часть 1: Вводная теория (кратко, детально — в лекции)

- 1. Хороший Dockerfile ≠ работающий Dockerfile.** Хороший image: быстро собирается, быстро скачивается, воспроизводимо пересобирается, безопасно запускается, легко диагностируется.
- 2. Слои (layers) и writable layer.** Image — это набор read-only слоёв; при запуске поверх добавляется writable layer контейнера. Изменения через `docker exec` живут только в этом слое и теряются после пересоздания контейнера. Отсюда — правило: *контейнер должен быть disposable*, состояние должно описываться кодом.
- 3. Build cache и порядок инструкций.** Стабильные и дорогие операции (установка зависимостей) — раньше; часто меняющиеся исходники — позже. Отсюда же `.dockerignore`: build context не должен включать `.git`, логи, `.env`, бинарники.
- 4. Multi-stage build.** Компилятор и build tools нужны тому, кто собирает. Пользователю приложения — нет. Второй stage — только runtime-минимум (для Go — это вообще просто статический бинарник на alpine).
- 5. PID 1, сигналы, graceful shutdown.** `docker stop` — это SIGTERM → grace period (по умолчанию 10 с) → SIGKILL. Приложение должно уметь корректно завершить работу. Exec form `CMD ["/app", "--flag"]` предпочтительнее shell form, чтобы приложение получало сигналы напрямую. Для Go: `signal.NotifyContext` + `http.Server.Shutdown`.
- 6. Контейнер ≠ маленькая VM.** Общий kernel host-a; изоляция — namespaces + cgroups; лимиты (`--memory`, `--cpus`) нужно задавать осознанно. Одна ответственность на контейнер: app, db и web-server — разные контейнеры.

7. Security hygiene. Не root без необходимости; `--privileged` — не «chmod 777»; `docker.sock` — почти административный API хоста; секреты не запекаются в image (`ENV PASSWORD=...` — плохо; BuildKit secret mounts — для build-time; переменные окружения/secrets — для runtime). Self-signed TLS-сертификат для стенда — исключение: сертификат генерируется на сборке через secret mount, а не хранится в git.

8. Compose глубже. DNS-имена сервисов вместо `localhost`; `healthcheck` + `depends_on: condition: service_healthy` (running ≠ healthy); `profiles` для необязательных сервисов; `docker compose config` — первая диагностическая команда.

9. Алгоритм диагностики. `docker ps -a` → `docker logs` → `docker inspect` → (died: exit code / events) или (alive: `stats`, `top`, сеть) → только потом `docker exec`. Знакомый минимум exit code: `0`, `1`, `125/126/127`, `137` (SIGKILL — не обязательно OOM).

Часть 2: Подготовка рабочего места

- Используется **control node** из ПЗ 1 (установлен Docker + Compose — это уже было сделано для Semaphore). Проверка:
`bash docker --version docker compose version docker buildx version # BuildKit`
должен быть доступен (нужен для secret mounts) Локальная разработка бэкенда также требует установленный Go 1.22+: `go version`.
- Убедитесь, что Semaphore/Ansible по-прежнему видит `web-server-1` по SSH (пригодится в следующей ПЗ).
- Создайте каталог проекта: `bash mkdir -p ~/devops-stack && cd ~/devops-stack git init` Всё, что вы напишете в этой ПЗ, должно оказаться в git — на следующем занятии этот репозиторий пойдёт в CI.

Структура репозитория (итоговая цель этой ПЗ):

```
devops-stack/
├── docker-compose.yml
├── .dockerignore
├── .env.example
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.js
│   └── src/...
├── backend/
│   ├── Dockerfile
│   ├── go.mod
│   ├── go.sum
│   ├── main.go          # HTTP-сервер: /api/notes, /health, graceful shutdown
│   ├── handlers.go      # хендлеры + model Note
│   ├── database.go      # подключение к PostgreSQL (pgxpool)
│   └── tests/
│       └── api_test.go  # unit-тесты (go test, httptest)
└── nginx/
    ├── Dockerfile      # openssl-builder → nginx:alpine runtime
    ├── conf.d/
    └── default.conf
```

Self-signed сертификат **не** попадает в git: он генерируется в отдельном build-слое (secret mount из локального файла `nginx/ssl/server.key`), а сам ключ хранится в `.dockerignore`.

Часть 3: Практика — Бэкенд (Go + PostgreSQL)

Шаг 1. Написание бэкенда.

Небольшое REST-API «заметки»: `GET /api/notes`, `POST /api/notes`, `GET /health`. Используется стандартный `net/http` + `pgx` (PostgreSQL) — никаких фреймворков.

```
// backend/go.mod
module notes

go 1.22

require (
    github.com/jackc/pgx/v5 v5.7.2
    github.com/jackc/pgx/v5/pgxpool v5.7.2
)
```

```
// backend/database.go
package main

import (
    "context"
    "log"
    "os"

    "github.com/jackc/pgx/v5/pgxpool"
)

var pool *pgxpool.Pool

func initDB() {
    dsn := os.Getenv("DATABASE_URL")
    if dsn == "" {
        dsn = "postgres://notes:notes@db:5432/notes"
    }
    ctx := context.Background()
    cfg, err := pgxpool.ParseConfig(dsn)
    if err != nil {
        log.Fatalf("parse DSN: %v", err)
    }
    pool, err = pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        log.Fatalf("connect: %v", err)
    }
    if err = pool.Ping(ctx); err != nil {
        log.Printf("DB not reachable at start: %v (will retry per request)", err)
    }
}

func ensureSchema(ctx context.Context) {
    _, err := pool.Exec(ctx, `
        CREATE TABLE IF NOT EXISTS notes (
            id      SERIAL PRIMARY KEY,
            title TEXT NOT NULL,
            body   TEXT NOT NULL DEFAULT ''
        );`)
    if err != nil {
        log.Printf("ensure schema: %v", err)
    }
}
```

```

// backend/handlers.go
package main

import (
    "context"
    "encoding/json"
    "io"
    "net/http"
    "strconv"
    "time"
)

type Note struct {
    ID      int64  `json:"id"`
    Title   string `json:"title"`
    Body    string `json:"body"`
}

func health(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    dbOK := pool.Ping(ctx) == nil
    status := "ok"
    if !dbOK {
        status = "degraded"
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]any{"status": status, "db": dbOK})
}

func listNotes(w http.ResponseWriter, r *http.Request) {
    rows, err := pool.Query(r.Context(), "SELECT id, title, body FROM notes ORDER BY id")
    if err != nil {
        http.Error(w, `{"error":"db error"}`, http.StatusInternalServerError)
        return
    }
    defer rows.Close()
    notes := []Note{}
    for rows.Next() {
        var n Note
        if err := rows.Scan(&n.ID, &n.Title, &n.Body); err != nil {
            http.Error(w, `{"error":"scan error"}`, http.StatusInternalServerError)
            return
        }
        notes = append(notes, n)
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(notes)
}

func addNote(w http.ResponseWriter, r *http.Request) {
    var n Note
    if err := json.NewDecoder(io.LimitReader(r.Body, 1<<20)).Decode(&n); err != nil {
        http.Error(w, `{"error":"invalid json"}`, http.StatusBadRequest)
        return
    }
    id, err := pool.QueryRow(r.Context(),

```

```

        "INSERT INTO notes (title, body) VALUES ($1, $2) RETURNING id",
        n.Title, n.Body).Scan()
    if err != nil {
        http.Error(w, `{"error":"db error"}`, http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(map[string]int64{"id": id})
}

func writeJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(code)
    json.NewEncoder(w).Encode(v)
}

func getNote(w http.ResponseWriter, r *http.Request) {
    idStr := r.PathValue("id")
    id, err := strconv.ParseInt(idStr, 10, 64)
    if err != nil {
        writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid id"})
        return
    }
    var n Note
    if err := pool.QueryRow(r.Context(), "SELECT id, title, body FROM notes WHERE id = $1",
        id).
        Scan(&n.ID, &n.Title, &n.Body); err != nil {
        writeJSON(w, http.StatusNotFound, map[string]string{"error": "not found"})
        return
    }
    writeJSON(w, http.StatusOK, n)
}

```

```
// backend/main.go
package main

import (
    "context"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    initDB()
    ensureSchema(context.Background())

    mux := http.NewServeMux()
    mux.HandleFunc("GET /health", health)
    mux.HandleFunc("GET /api/notes", listNotes)
    mux.HandleFunc("POST /api/notes", addNote)
    mux.HandleFunc("GET /api/notes/{id}", getNote)

    srv := &http.Server{
        Addr:         ":8000",
        Handler:      mux,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
    }

    // Graceful shutdown: SIGTERM/SIGINT → http.Server.Shutdown
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGTERM, syscall.SIGINT)
    defer stop()

    go func() {
        log.Println("notes-api listening on :8000")
        if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
            log.Fatalf("listen: %v", err)
        }
    }()

    <-ctx
    log.Println("SIGTERM received, shutting down gracefully...")
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil {
        log.Printf("graceful shutdown failed: %v", err)
        os.Exit(1)
    }
    log.Println("server stopped")
}
```

Тесты (появятся полноценно в ПЗ 3, здесь — только скелет):

```
// backend/tests/api_test.go
package main

import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestHealthEndpointExists(t *testing.T) {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /health", health)

    req := httptest.NewRequest(http.MethodGet, "/health", nil)
    rec := httptest.NewRecorder()
    mux.ServeHTTP(rec, req)

    if rec.Code != http.StatusOK {
        t.Fatalf("status = %d, want 200", rec.Code)
    }
    // тело должно содержать поле "status"
    if !contains(rec.Body.String(), `"status"`) {
        t.Fatalf("body %q missing 'status'", rec.Body.String())
    }
}

func contains(s, sub string) bool {
    return len(s) >= len(sub) && (s == sub || len(sub) == 0 || (func() bool {
        for i := 0; i+len(sub) <= len(s); i++ {
            if s[i:i+len(sub)] == sub {
                return true
            }
        }
        return false
    })())
}
```

Запуск тестов локально: `cd backend && go test ./tests/... -v` (тест не требует живой БД — health отрабатывает и в degraded-состоянии).

Шаг 2. «Плохой» Dockerfile — соберите и оцените.

Намеренно кривой вариант (из лекции) — соберите его, посмотрите на размер и время:

```
FROM golang:1.22
WORKDIR /app
COPY . .
ENV DATABASE_URL=postgres://notes:notes@localhost:5432/notes
RUN go mod download && go build -o /app/notes .
CMD ./notes
```

Назовите как минимум 5 проблем (подсказка: сборка на полном `golang` без multi-stage — runtime ~1 GB+, секрет в ENV, shell form CMD, сборка от root, исходники и модули в финальном образе, отсутствие graceful shutdown-согласованности с PID 1...).

Шаг 3. Хороший Dockerfile (multi-stage).

```
# backend/Dockerfile
FROM golang:1.22-alpine AS builder
WORKDIR /src
# Кэш модулей: зависимости ставятся до исходников
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /out/notes .

FROM alpine:3.20 AS runtime
RUN adduser -D -H app
WORKDIR /app
COPY --from=builder /out/notes ./notes
RUN chown app:app /app
USER app
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=5 CMD wget -qO- http://127.0.0.1:8000/health || exit 1
# Exec form: бинарник — PID 1 и получает SIGTERM напрямую
CMD ["/app/notes"]
```

`.dockerignore` (в корне backend/):

```
.git
bin/
*.test
.env
```

Проверки:

```
cd backend
docker build -t notes-api . # сборка 1
docker build -t notes-api . # сборка 2 — должны быть CACHED-слои
# поменяйте handlers.go, пересоберите — слои go mod download не должны пересобиаться
docker image ls | grep notes-api # alpine-runtime — сравните размер с «плохим»
docker image history notes-api
```

Запуск с БД временно (для проверки, final-стек ниже):

```
docker network create -d notes-net
docker run -d --name pg-test --network notes-net -e POSTGRES_USER=notes -e
  POSTGRES_PASSWORD=notes -e POSTGRES_DB=notes postgres:16
docker run --rm --network notes-net -e DATABASE_URL=postgres://notes:notes@pg-test:5432/
  notes -p 8000:8000 notes-api
docker exec -it pg-test psql -U notes -c "select 1;"
curl http://localhost:8000/health
docker rm -f pg-test && docker network rm notes-net
```

Часть 4: Практика — Фронтенд (Vite + multi-stage)

Шаг 1. Любой SPA. Самый простой вариант — Vite + React, либо чистый JS. Ключевое для этой ПЗ — сборка «артефакт + сервер, который его отдаёт» в одном контейнере:

```
cd frontend
npm create vite@latest . -- --template react
npm install
```

Минимальный `App.js` : страница с формой (title/body) и списком заметок, дёргает `/api/notes` (на стенде его проксирует nginx на бэкенд) и показывает статус `/health`.

Шаг 2. Multi-stage Dockerfile для фронтенда.

```
# frontend/Dockerfile
FROM node:22-alpine AS builder
WORKDIR /src
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine AS runtime
COPY --from=builder /src/dist /usr/share/nginx/html
# k8s-совместимый healthcheck уже зададим в compose
EXPOSE 80
```

Обратите внимание на паттерн из лекции: `COPY package*.json` → `npm ci` → `COPY . .` → `npm run build`. Build tools (node, исходники) остаются в stage `builder` и **не попадают** в runtime-образ.

```
docker build -t notes-web .
docker run --rm -p 8080:80 notes-web # http://localhost:8080
docker image ls | grep notes-web    # сравните размер с backend-образом
```

Часть 5: Практика — nginx-шлюз (self-signed TLS) + Compose-стек

Цель: один `docker compose up` поднимает весь стек; сервисы знают друг друга по DNS; readiness-зависимости через healthcheck; секреты не в файле compose; внешний трафик — только через nginx по HTTPS (self-signed сертификат, сгенерированный на сборке, не хранится в git).

Шаг 1. Генерация self-signed сертификата (локально, в build-слое).

```
# nginx/generate-cert.sh — выполняется при локальной разработке / перед docker build
mkdir -p nginx/ssl
openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout nginx/ssl/server.key \
  -out nginx/ssl/server.crt \
  -subj "/CN=devops-stack.local" \
  -addext "subjectAltName=DNS:devops-stack.local,DNS:localhost,IP:127.0.0.1"
chmod 600 nginx/ssl/server.key
```

В git попадает **только** `generate-cert.sh` и `conf.d/default.conf`; `ssl/` добавляется в `.dockerignore` и `.gitignore`. На стенде сертификат существует только внутри образов (build

secret) — это допустимо, т.к. сертификат self-signed и не является долгоживущим секретом; но **ключи/пароли БД** по-прежнему только в `.env/secrets`.

Шаг 2. Конфигурация nginx.

```
# nginx/conf.d/default.conf
upstream notes_backend {
    server backend:8000;
    keepalive 32;
}

server {
    listen 8443 ssl;
    server_name devops-stack.local localhost;

    # TLS: сертификаты из build-слоя (secret mount → /etc/nginx/ssl)
    ssl_certificate      /etc/nginx/ssl/server.crt;
    ssl_certificate_key  /etc/nginx/ssl/server.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers           HIGH:!aNULL:!MD5;
    ssl_session_cache    shared:SSL:10m;
    ssl_session_timeout  10m;

    # SPA: фронтенд отдаёт nginx-frontend
    location / {
        proxy_pass http://frontend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # API: бэкэнд
    location /api/ {
        proxy_pass http://notes_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 3s;
        proxy_read_timeout 30s;
    }

    # Healthcheck для orchestrator / compose
    location /healthz {
        access_log off;
        return 200 "ok\n";
        add_header Content-Type text/plain;
    }
}
```

Шаг 3. Dockerfile nginx (openssl-builder → runtime).

```
# nginx/Dockerfile
# Stage 1: генерация self-signed сертификата (secret mount – ключ не запечатывается в слои)
FROM alpine:3.20 AS cert-builder
RUN apk add --no-cache openssl
WORKDIR /certs
# --mount=type=secret: содержимое появляется только на время этого RUN
RUN --mount=type=secret,id=cert_key,target=/tmp/server.key \
  --mount=type=secret,id=cert.crt,target=/tmp/server.crt \
  cp /tmp/server.key /etc/nginx-ssl/server.key \
  && cp /tmp/server.crt /etc/nginx-ssl/server.crt \
  && chmod 600 /etc/nginx-ssl/server.key

# Stage 2: runtime
FROM nginx:1.27-alpine AS runtime
RUN rm -f /etc/nginx/conf.d/default.conf
COPY conf.d/default.conf /etc/nginx/conf.d/default.conf
# Копируем только сертификат+ключ из cert-builder (build-артефакт, не runtime-слои)
COPY --from=cert-builder /etc/nginx-ssl/ /etc/nginx/ssl/
RUN chown -R nginx:nginx /etc/nginx/ssl && chmod 644 /etc/nginx/ssl/server.crt && chmod 600 /
  etc/nginx/ssl/server.key
EXPOSE 8443
HEALTHCHECK --interval=10s --timeout=3s --retries=5 CMD wget -q0- https://127.0.0.1:8443/
  healthz --no-check-certificate || exit 1
```

Сборка с BuildKit secrets (запускайте **корнем репозитория**, т.к. secret-файлы лежат в `nginx/ssl/`):

```
cd nginx/
DOCKER_BUILDKIT=1 docker build \
  --secret id=cert_key,src=ssl/server.key \
  --secret id=cert.crt,src=ssl/server.crt \
  -t notes-nginx .
```

Шаг 4. Сведение в Compose-стек.

```

# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: notes
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD not set}
      POSTGRES_DB: notes
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U notes -d notes"]
      interval: 5s
      timeout: 3s
      retries: 10
      restart: unless-stopped

  backend:
    build:
      context: ./backend
    depends_on:
      db:
        condition: service_healthy      # не «запущен», а «готов принимать соединения»
    environment:
      DATABASE_URL: postgres://notes:${DB_PASSWORD}@db:5432/notes
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://127.0.0.1:8000/health || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5
      restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 256M
          cpus: "1"

  frontend:
    build: ./frontend
    depends_on:
      - backend
    restart: unless-stopped

  nginx:
    build:
      context: ./nginx
      # BuildKit secrets: сертификат не попадает в git, а используется только на сборке
      # (запуск через docker build с --secret см. выше; здесь – только для локального dev)
    depends_on:
      - backend
      - frontend
    ports:
      - "8443:8443"      # https://localhost:8443
    restart: unless-stopped

# Необязательный сервис – доступный только через profile
adminer:
  image: adminer

```

```

    profiles: ["debug"]
    ports:
      - "8081:8080"

volumes:
  db_data:

```

Примечание про secrets в compose: `docker compose` сам не передаёт `--secret` в `build`. Для этого есть два варианта: (1) собрать `nginx`-образ отдельно `docker build --secret ...` и в `compose` указать `image: notes-nginx` вместо `build:`, (2) использовать BuildKit `docker compose build` с секцией `x-build:` / `build.secrets`. На этой ПЗ достаточно варианта 1 — соберите образ руками, затем `docker compose up -d`.

```

# .env.example — шаблон для git; реальный .env НЕ коммитится
DB_PASSWORD=change-me-strong-password

```

Работа со стеком:

```

# 1. Сгенерируйте сертификат (один раз)
cd nginx && bash generate-cert.sh && cd ..

# 2. Соберите nginx-образ с секретами
cd nginx
DOCKER_BUILDKIT=1 docker build \
  --secret id=cert_key,src=ssl/server.key \
  --secret id=cert.crt,src=ssl/server.crt \
  -t notes-nginx .
cd ..

# 3. Замените build: на image: notes-nginx в docker-compose.yml (или используйте
    compose.override.yml)

# 4. Поднимите стек
cp .env.example .env && $EDITOR .env      # задать свой пароль
docker compose config                      # <- первая команда диагностики
docker compose up -d
docker compose ps                          # статусы: started, healthy, unhealthy
docker compose logs -f backend             # логи в stdout/stderr — по замыслу

# Проверка через nginx (self-signed TLS — -k для пропуска проверки сертификата):
curl -k https://localhost:8443/health      # {"status":"ok","db":true}
curl -k https://localhost:8443/api/notes # []
curl -k -X POST https://localhost:8443/api/notes \
  -H 'Content-Type: application/json' \
  -d '{"title":"first","body":"hello"}'
curl -k https://localhost:8443/api/notes # [{"id":1,"title":"first","body":"hello"}]

# SPA в браузере: https://localhost:8443 (примите self-signed предупреждение)

docker compose --profile debug up -d      # поднимает adminer
docker compose --profile debug stop adminer # убирает его
docker compose down

```

Задания для самопроверки (обязательные):

1. Сломать намеренно и диагностировать **по алгоритму из лекции** (не «перезапустить и забыть»):
 - Установить `DATABASE_URL=...@localhost:5432/...` в backend — записать, что показывает `docker compose ps`, `logs`, как проявляется ошибка. Исправить на `db`.
 - Убрать healthcheck у `db` и `condition: service_healthy` — наблюдать race condition (backend стартует раньше, чем Postgres готов). Вернуть.
 - Задать `deploy.resources.limits.memory: 64M` у backend — наблюдать OOM/exit 137; записать, что показали `docker ps -a`, `docker inspect`, `docker events`.
2. Проверить graceful shutdown: отправить SIGTERM (`docker stop backend`) и показать по логам, что Go-сервер корректно завершает активные соединения (`SIGTERM received, shutting down gracefully...`), а не получает SIGKILL.
3. `docker top backend` — показать, что PID 1 это бинарник `/app/notes` (exec form CMD), а не `sh -c`.
4. Проверить TLS-цепочку: `openssl s_client -connect localhost:8443 -showcerts` — убедиться, что сертификат self-signed (self-signed certificate).
5. `docker system df` / `docker system prune` (без `--volumes`!) — показать, что и где съедает место.

Дополнительные задания (на оценку):

1. Заменить inline healthcheck бэкенда на отдельный healthcheck-скрипт и сравнить подходы.
2. Использовать BuildKit secret mount для «секрета сборки» фронтенда (например, версия релиза, прошиваемая в `npm run build`).
3. Написать второй Compose-файл (`docker-compose.override.yml`) с отладочными настройками (nginx без TLS, прямой порт backend) для локальной разработки и объяснить разницу.
4. Развернуть **второй экземпляр стека** на `web-server-1` из ПЗ 1 (скопировать репозиторий по scp/git, `docker compose up -d`) и сделать, чтобы оба экземпляра работали независимо.
5. Добавить **redirect http → https** на nginx (порт 80 → 301 → 8443) и обновить SPA для работы с HTTPS.

Часть 6: Проверка и критерии успешного выполнения

- Репозиторий `devops-stack` в git: `frontend/`, `backend/`, `nginx/`, `docker-compose.yml`, `.dockerignore`, `.env.example`; в git **нет** `.env`, `.git`-мусора, секретов в Dockerfile, **нет** `ssl/` (только `generate-cert.sh`).
- `docker compose up -d` с нуля поднимает работающий стек (frontend → nginx(TLS) → backend → postgres) без ручных вмешательств.
- Оба Dockerfile (backend, frontend) — multi-stage, зависимости ставятся до исходников, есть `.dockerignore`, app не root, exec form CMD.
- nginx-образ: self-signed сертификат в `/etc/nginx/ssl/`, `listen 8443 ssl`, `ssl_certificate` + `ssl_certificate_key` заданы, `healthcheck` через `wget --no-check-certificate`.

- `curl -k https://localhost:8443/health` → `{"status":"ok","db":true}`.
- `curl -k https://localhost:8443/api/notes` → JSON (через nginx → backend → postgres).
- SPA доступен по `https://localhost:8443` (self-signed, примите предупреждение).
- Healthchecks у `db` и `backend`; `depends_on` с `condition: service_healthy`.
- Ресурсные лимиты у `backend`; продемонстрирован и объяснён сценарий OOM (exit 137).
- Диагностика каждой намеренной поломки пройдена по алгоритму `ps -a → logs → inspect → stats/top → exec`, и записаны выводы.
- Graceful shutdown: `docker stop backend` → логи показывают `SIGTERM received, shutting down gracefully...`.
- (доп.) Второй экземпляр стека работает на `web-server-1`, развёрнут с репозитория.

К чему ведёт следующая ПЗ: этот репозиторий — готовый «продукт» для CI/CD. На практике №3 мы подключим к нему GitHub Actions (или GitLab CI / Argo CD — на выбор курса): линтинг (`golangci-lint`), unit-тесты (`go test`), сборка образов с multi-stage, push в registry, деплой стека на `web-server-*` **через Semaphore** — то есть связка Ansible + Docker + CI/CD без Kubernetes.